

AGENTIC AI FUNDAMENTALS

What is AGENTIC AI

Agentic AI is a branch of artificial intelligence focused on building autonomous, intelligent agents capable of making decisions, interacting with other agents and completing complex tasks with minimal human intervention. It combines LLMs, multi-agent systems and workflow orchestration to build advanced AI applications.

The important word here is **agency**. **AGENCY MEANS THAT THE SYSTEM IS NOT RESTRICTED TO PRODUCING A SINGLE RESPONSE. IT CAN PARTICIPATE IN A SEQUENCE OF DECISIONS AND ACTIONS DIRECTED TOWARD AN OBJECTIVE.**

Consider the request:

"Find the cause of the production API failures and prepare a report."

- A conventional generative AI system might explain common reasons APIs fail or generate debugging suggestions.
- An agentic system could potentially inspect monitoring data, search logs, identify suspicious services, query documentation, run permitted diagnostic tools, compare evidence, formulate a hypothesis, test it, and produce a report.

THE DIFFERENCE IS THEREFORE NOT SIMPLY THAT AGENTIC AI IS "MORE INTELLIGENT." THE IMPORTANT DIFFERENCE IS THE CONTROL LOOP AROUND THE MODEL.

A simplified architecture looks like:

Goal → Reason/Plan → Act → Observe → Evaluate → Adjust → Act again → Finish

How Agentic AI Works

Agentic AI systems operate through various steps such as:

- **Perception:** *Collects relevant real-time data* and retrieves past information from memory to provide context for the task.
- **Reasoning:** *Interprets inputs using domain-specific knowledge* and patterns stored in memory or knowledge bases.
- **Goal Setting:** *Defines clear objectives* and creates a focused plan based on input and past outcomes.
- **Decision-Making:** *Chooses actions based on efficiency*, accuracy and safety using past successful strategies.
- **Execution:** *Performs tasks using tools like APIs* and records results for future reference.
- **Learning and Adaptation:** Improves over time by learning from feedback and storing useful experiences.
- **Orchestration:** *Collaborates with other agents* in multi-agent systems to complete tasks efficiently.

Core characteristics of Agentic AI

Agentic systems commonly combine several capabilities.

Goal orientation means the system is given an objective rather than a complete sequence of instructions. For example, "Resolve this customer support issue" leaves the system to determine intermediate actions.

Planning and decomposition allow a large task to be converted into smaller tasks. An agent asked to analyze a software repository might decide to inspect the directory structure, identify important modules, examine dependencies, run tests, investigate failures, and summarize its findings.

Tool use allows the system to interact with external environments. Tools might include APIs, databases, web search, terminals, code interpreters, browsers, calendars, email systems, or internal enterprise services.

Memory and state allow information from earlier steps to influence later decisions. Without state, every interaction effectively begins from scratch.

Reasoning and decision-making allow the system to determine what action should happen next rather than following only a predefined sequence.

Reflection or evaluation allows the system to examine whether an action produced the expected result. For example, if generated code fails a test, the agent can inspect the error and attempt another solution.

Autonomy refers to how much of this process occurs without requiring a human to approve every intermediate action. Autonomy is usually bounded rather than absolute.

AI AGENT

*An **AI Agent** is an autonomous software system powered by a foundational Artificial Intelligence model (like an LLM) that can **perceive its environment, reason through complex goals, and take independent action using external tools to achieve a specific outcome.***

Unlike a standard chatbot that simply responds to a single text prompt with static text, an AI Agent behaves like a virtual employee. You give it a high-level goal, and it independently determines the steps, gathers the data, and executes the tasks required to finish the job.

The 4 Core Components of an AI Agent

To turn a standard AI model into an autonomous agent, an orchestration framework wraps four essential components around it:

1. **The Brain (Foundation Model):** The central engine (like Anthropic Claude or OpenAI GPT). It provides the core cognitive abilities, language understanding, and logical reasoning needed to make decisions.
2. **Planning & Orchestration:** The logical framework (like the *ReAct framework*) that allows the agent to break a massive goal into smaller, sequential steps. It constantly evaluates its own progress: *"I completed step 1, here is what I learned, now I need to do step 2."*

3. **Memory:** The ability to retain information. This is split into **short-term memory** (keeping track of the context within an active conversation) and **long-term memory** (remembering user preferences, past interactions, or historical files across weeks or months).
4. **Tools & APIs:** The physical "hands and feet" of the agent. This includes web browsers, calculators, database connectors, code execution sandboxes, or software APIs (like Slack, Jira, or Salesforce) that allow the agent to affect the real world.

A useful abstraction is:

Agent = Model + Instructions + State/Memory + Tools + Control Loop

Not every implementation needs every component in exactly this form, but this architecture is useful for understanding modern LLM-based agents.

The **model** provides capabilities such as language understanding, reasoning, classification, or generation.

The **instructions** define objectives, constraints, policies, roles, and expected behavior.

The **tools** give the agent capabilities beyond text generation. Examples include `search_web()`, `query_database()`, `execute_code()`, `send_email()`, or `create_ticket()`.

The **state or memory** stores information required across steps.

The **control loop** determines when the model should reason, invoke a tool, process the result, continue working, ask a human for information, or terminate.

This last component is particularly important. Simply giving an LLM access to an API does not automatically produce a robust agent. The orchestration around the model determines much of the system's reliability.

TYPES OF AGENTS

1. Single-Loop / Tool-Calling Agent

This is the entry-level agent archetype. It sits in a simple "Reason-and-Act" (ReAct) loop. You give it a goal and a toolbox; it looks at the goal, picks a tool, looks at the result, and repeats until it has an answer.

- **Best Framework:** **LangChain** (`create_openai_tools_agent`).
- **Use Case:** A research bot that searches Google, calculates a financial formula, and prints a summary.

2. Sequential / Linear Agent Pipeline

In this design, tasks are executed in a strict, predefined chain. One agent completes its specific job and passes its output directly to the next agent down the assembly line.

- **Best Framework: CrewAI** (Sequential Process).
- **Use Case:** An article writing pipeline: Agent 1 researches a topic → Agent 2 drafts the copy → Agent 3 proofreads the text.

3. Hierarchical / Manager-Led Agent Crew

This pattern mimics a corporate office structure. You build an autonomous **Manager Agent** that does not execute tools itself. Instead, it receives the high-level user goal, breaks it into tasks, assigns those tasks to specialized **Worker Agents**, and reviews their work before finalizing.

- **Best Framework: CrewAI** (Hierarchical Process).
- **Use Case:** A software development crew where a Product Manager Agent assigns coding tasks to a Developer Agent and code review to a QA Agent.

4. State-Machine / Graph-Based Agent

The most complex and powerful pattern. Instead of a linear line, you map your agent logic as a visual flowchart (nodes and edges). You code explicit conditions: *"If the QA check fails, route the data backward 3 steps to the Developer Agent; if it passes, route it forward to production."*

- **Best Framework: LangGraph** (an extension of LangChain).
- **Use Case:** Customer support systems that require strict business logic guardrails and looping workflows.

ORCHESTRATION

Orchestration is the central management engine—the "master conductor"—that coordinates, guides, and connects an AI model to the external systems, memory modules, and tools required to execute an end-to-end business process.

On its own, a Large Language Model (LLM) is just a text predictor; it has no inherent ability to look up data in a database, remember past human conversations, or send an email.

Orchestration software wraps around the model to turn its raw text predictions into actionable workflows.

The Core Responsibilities of an Orchestrator

An orchestrator operates as a continuous state-machine loop that handles four foundational tasks:

ORCHESTRATION ENGINE

 1. Prompt Construction & System Rules	
 2. State & Memory Routing (Short/Long)	

	 3. Tool Execution & API Call Intercepts	
	 4. Multi-Agent Routing & Conditional Steps	

1. **State and Memory Routing:** It dynamically saves active conversation logs (short-term memory) and injects historical corporate databases or user profiles (long-term memory) into the prompt window before the model evaluates a request.
2. **Tool Execution Handling:** When a model prints a text command indicating it needs to use a tool (e.g., Action: call_weather_api(city="Vancouver")), the orchestrator catches this intent, pauses model text generation, executes the live software code, and feeds the resulting payload back into the model's token window.
3. **Flow Control & Guardrails:** It enforces strict programmatic boundaries. For example, it ensures an AI agent loops backward to fix a broken code block rather than moving forward blindly, or it blocks forbidden keywords before they exit the system.
4. **Multi-Agent Coordination:** In complex environments, it manages the assembly line, passing data payloads systematically from a *Researcher Agent* to a *Writer Agent* once specific quality benchmarks are satisfied.

Orchestration Types: Open-Source vs. Cloud-Native

Depending on how you design your application infrastructure, orchestration is typically handled via one of two paradigms:

- **Code-Driven Open-Source:** Frameworks like **LangChain**, **LangGraph**, and **CrewAI**. Developers write the orchestration loops explicitly in Python or TypeScript, giving them infinite control over memory states and local tool execution logic.
- **Fully Managed Cloud-Native:** Services like **Agents for Amazon Bedrock** or **Google Cloud Vertex AI Agent Builder**. The cloud platform manages the underlying orchestration loops, infrastructure, and serverless executions behind the scenes, leaving developers to simply connect API schemas and text instructions via a console interface.

Chain of Thought vs. Agent Orchestration

While both concepts involve step-by-step thinking, they operate at entirely different layers of execution:

Feature / Metric	Chain of Thought (CoT)	Agent Orchestration
Primary Level	Cognitive Technique: A prompting or inference strategy used <i>inside</i> a language model.	System Architecture: A structural framework built <i>around</i> a language model.
How it Works	The model outputs its internal reasoning process textually ("Let's think step-by-step") before generating a final response.	A wrapper loop that combines the model's reasoning with external APIs, memory updates, and tool execution .

Interaction	Static/Linear: It cannot pause its thoughts to look up new information midway through generating text. Dynamic/Interactive: It can stop, call an external tool, ingest new data, and rewrite its plan based on real-world results.
--------------------	---

ReAct FRAMEWORK

*The **ReAct framework** (short for **Reason and Act**) is an architectural prompting strategy and cognitive design pattern that enables Large Language Models (LLMs) to solve complex tasks by combining structured reasoning with tool execution.*

Introduced in 2022 by researchers from Princeton University and Google DeepMind, it serves as the foundational design pattern for most modern, single-loop **AI agents** built using frameworks like LangChain, CrewAI, and Amazon Bedrock.

The **ReAct Orchestration Loop** is the engine that drives autonomous AI agents. Developed by researchers at Princeton and Google DeepMind, *ReAct stands for Reasoning and Acting*.

It is a structured execution cycle that forces a Large Language Model (LLM) to alternate between **thinking out loud** (Reasoning) and **executing software code** (Acting). This loop keeps running iteratively until the agent solves the user's problem.

The Core Stages of the Loop

The loop functions as a strict, repeating lifecycle managed by an orchestration engine (like LangChain or Amazon Bedrock):

1. Thought (Reasoning)

The model looks at the user's goal and **writes out its internal reasoning process** in plain text. It explains *why* it cannot answer the question instantly and determines exactly what information it needs to gather next.

2. Action (Acting)

The model reaches a point where it needs external data. It halts regular text generation and outputs a structured **tool-calling command** containing technical parameters (e.g., Action: WebSearch["weather in Vancouver"]).

3. Observation (Learning)

The orchestration framework intercepts this command, pauses the LLM, and runs the actual software tool (like executing an API call or querying a SQL database). The raw data returned by that tool is called the **Observation**. The orchestrator injects this text observation right back into the model's token history.

Tracing a Live Loop: A Technical Example

Imagine an AI agent equipped with a **Web Search Tool** and a **Calculator Tool**. The user asks: *"What is the current population of Canada multiplied by 1.5?"*

Here is exactly how the ReAct loop executes behind the scenes:

- **THOUGHT 1:** *I need to find the current population of Canada first. I don't have real-time internet data in my static training weights, so I must use a search engine tool.*
- **ACTION 1:** WebSearch[current population of Canada 2026]
- **OBSERVATION 1:** *(The orchestrator runs the web search and returns: "Statistics Canada reports the population has reached 42,000,000.")*
- **THOUGHT 2:** *I now have the population (42,000,000). The user wants me to multiply this number by 1.5. To avoid making a mental math error, I should use the calculator tool.*
- **ACTION 2:** Calculator[42000000 * 1.5]
- **OBSERVATION 2:** *(The orchestrator executes the math code locally and returns: "63000000")*
- **THOUGHT 3:** *I have successfully found the population and calculated the multiplied result. I have all the data required to answer the user.*
- **FINAL ANSWER:** The current population of Canada multiplied by 1.5 is **63,000,000**.

Why is this Loop Necessary?

1. **Prevents Hallucinations:** LLMs are notorious for confidently guessing facts or making basic math errors. ReAct forces them to look up fresh data and offload computation to accurate software plugins.
2. **Self-Correction:** If **Action 1** returns an error (e.g., a broken URL or a missing database table), the model reads the error in the **Observation** stage. It can then generate **Thought 2** to adjust its approach: *"My last search failed. Let me try searching with a different keyword phrase."*
3. **Auditable Logic:** Because the model outputs its thoughts explicitly, human developers can review the logs to see *exactly* why an agent made a decision or where an integration went wrong.

AI Agent Frameworks

AI agent frameworks **ARE TOOLS AND PLATFORMS THAT HELP DEVELOPERS BUILD INTELLIGENT AGENTS** capable of performing tasks autonomously, making decisions and interacting with users or systems. These frameworks simplify the development of agent-based applications by providing components for reasoning, memory and tool integration.

1. LangGraph

[LangGraph](#) is a framework used to build advanced AI agents that handle complex, stateful workflows. It uses graph-based execution, enabling step-by-step and branching logic for better control over agent behavior.

2. AutoGen

[AutoGen](#) is a Microsoft-backed framework for building multi-agent systems where AI agents collaborate with each other and humans to automate complex tasks through dynamic conversations.

- **Multi-Agent Conversations:** Enables agents to interact autonomously or with human input for collaboration, debate and problem solving.
- **Role-Based Agents:** Agents can be assigned specific roles, personas and responsibilities.
- **Flexible Interaction Flow:** Supports both fully automated and human-in-the-loop workflows.
- **Task-Oriented Collaboration:** Agents coordinate to break down and solve complex tasks step-by-step.

3. CrewAI

[CrewAI](#) is an open-source framework for building teams of AI agents that collaborate with defined roles to solve complex, multi-step tasks such as research, content creation and business workflows.

- **Role-Based Agents:** Agents are assigned roles, goals and backstories for specialized behavior.
- **Task Coordination:** Tasks are defined and distributed among agents for efficient execution.
- **Collaborative Execution:** Multiple agents work together, optionally with human input, to complete complex workflows.

4. Semantic Kernel

[Semantic Kernel](#) is an open-source SDK by Microsoft that integrates large language models into existing applications, enabling structured orchestration of AI, code and external services.

- **AI + Code Integration:** Combines traditional programming logic with LLM capabilities.

5. LangChain

[LangChain](#) is an open source framework that simplifies building applications using large language models by connecting prompts, data, memory and tools into structured workflows.

- **Tool Integration:** Connects with APIs, databases and search engines to enhance capabilities.
- **Human-in-the-Loop:** Supports workflows with human review and intervention.
- **Agent Workflows:** Enables creation of agents that can think, act and collaborate.
- **LLM Integration:** Works with models like OpenAI, Anthropic and Hugging Face.
- **Memory and Context:** Maintains conversation history for better and personalized responses.

6. No-Code AI Agent Platforms

No-code and visual AI platforms allow users to build AI agents and workflows using simple drag and drop interfaces, making development faster and accessible without coding.

A. n8n

[n8n](#) is a workflow automation tool that **COMBINES AI WITH APP INTEGRATIONS USING A VISUAL INTERFACE**.

- **Workflow Automation:** Build AI workflows by connecting apps like Notion, Stripe and Google Sheets.
- **Trigger-Based Flows:** Start workflows using events like webhooks, messages or custom triggers.
- **Business Use Cases:** Automate tasks like CRM updates, lead management and communication.

B. Langflow

[Langflow](#) is a visual, drag and drop framework for building and testing AI workflows, including multi agent systems and RAG based applications.

- **Multi Agent and RAG Support:** Enables building complex workflows with multiple agents and knowledge based retrieval.
- **Model Flexibility:** Works with various LLMs and vector databases, including open source options.
- **Fast Prototyping:** Allows quick testing, iteration and deployment of workflows.
- **Easy Integration:** Flows can be shared, embedded or connected via APIs.

Choosing the Right Framework

1. Task Complexity

- LangGraph and CrewAI are best for multi step, stateful workflows.
- LangChain works well for modular, LLM based tasks like Q/A and RAG.
- AutoGen is ideal for dynamic multi agent conversations and collaboration.

2. Integration Needs

- Langflow and n8n are great for no code, visual workflows and quick integrations.
- Semantic Kernel is better for deep integration into applications using code.

3. Customizability and Scalability

- Semantic Kernel, LangGraph and AutoGen are all built for enterprise scale extensibility, offering modular plugins, agent frameworks and support for advanced orchestration.
- CrewAI is particularly suited for collaborative multi agent teams in specialized roles, ideal for automating business processes that benefit from teamwork.

4. Privacy and Data Security

- Semantic Kernel and LangGraph are strong for secure and compliant systems.
- AutoGen and CrewAI require careful design for secure communication.
- Langflow and n8n are convenient but need validation for secure integrations.

Building AI Agents

Building AI agents means creating software that can perform tasks intelligently and autonomously. This involves defining what the agent should do, training it with data and using technologies like machine learning and natural language processing. The goal is to make the agent capable of understanding instructions, learning from experience and improving its performance over time.

Core Principles for Building AI Agents

- **Start Simple:** Begin with the least complex solution often a single LLM call or a basic workflow. Only introduce agentic complexity when the task requires more flexibility or dynamic decision-making.

- **Agent vs. Workflow:** Use agents for open-ended, dynamic tasks where the steps aren't predetermined. Use workflows for structured, predictable processes.
- **Composable Patterns:** Build agents from simple, composable workflow patterns, which can be combined as needed for more complex applications.

AI AGENT, AGENTIC workflow, and AGENTIC AI system

The terms **AI agent**, **agentic workflow**, and **Agentic AI system** are closely related, but they describe different levels of architecture. The easiest distinction is: **an agent is an autonomous decision-making component**, **an agentic workflow is the process in which AI makes some dynamic decisions**, and **an Agentic AI system is the complete system that brings agents, workflows, tools, memory, guardrails, and infrastructure together**.

1. AI Agent

Definition

An **AI agent is a software component that receives a goal, observes relevant information, decides what action to take next, performs actions through available tools, and uses the results of those actions to continue toward the goal**.

The important characteristic is that the **agent has some control over what happens next**. The developer does not necessarily specify every step beforehand.

For example, suppose we give a coding agent this goal:

"Find and fix the reason the authentication tests are failing."

The agent might decide to inspect the test failures, search the repository, read authentication code, modify a file, run the tests again, inspect the new results, and continue until it reaches a termination condition.

Conceptually:

Goal → Observe → Decide → Act → Observe → Decide → Act → ... → Result

The agent is therefore the **decision-making entity** inside the architecture.

A useful simplified formula is:

Agent = Model + Instructions + State + Tools + Decision/Control Loop

An LLM often provides the reasoning capability in modern agents, but an agent does not inherently require an LLM.

2. Agentic Workflow

Definition

An **agentic workflow is a structured process that contains one or more AI-controlled decision points while still keeping much of the overall process predefined or deterministic**.

The distinction here is important. Not every step has to be controlled by an autonomous agent.

Consider a customer-support workflow:

Receive ticket → Classify ticket → Retrieve customer data → Determine solution → Human approval → Send response → Update ticket

Some steps can be ordinary deterministic software. For example, updating the ticket after approval might always happen in exactly the same way.

Other steps can be agentic. The AI might decide what information it needs, which knowledge source to search, whether additional customer information is required, or whether the issue should be escalated.

Therefore, an agentic workflow might look like:

Deterministic Step → AI Decision → Tool → Deterministic Validation → AI Decision → Human Approval → API Action

The system has **bounded flexibility** rather than giving an agent complete control over the entire process.

This approach is common in production because deterministic workflows provide predictability, while AI-controlled steps provide flexibility where rigid rules are insufficient.

3. Agentic AI System

Definition

An **Agentic AI system** is the complete software system designed around agents or agentic behavior to accomplish goals through planning, reasoning, tool use, state management, feedback, and controlled autonomous actions.

It is broader than a single agent.

For example, imagine a software-development platform that automatically handles GitHub issues. The complete system might contain a **planner agent** that analyzes the issue, a **coding agent** that modifies source code, a **review agent** that checks the changes, repository-search tools, Git tools, test runners, short-term state, persistent memory, permission controls, human approval gates, logging, monitoring, retry mechanisms, and security guardrails.

All of these components together constitute the **Agentic AI system**.

Conceptually:

Agentic AI System

- Agents
- Agentic workflows
- LLMs/models
- Tools and APIs
- State and memory
- Databases
- Planning mechanisms
- Human approval mechanisms
- Guardrails and permissions
- Evaluation

- Monitoring and observability
- Retry and recovery mechanisms

Therefore, the Agentic AI system represents the **complete architecture**, not merely the LLM or agent.

The Difference in One Example

Consider an AI system designed to resolve GitHub issues.

An **AI agent** might receive: "Investigate why this unit test is failing."

It independently decides which files to inspect, which commands to execute, and what changes to attempt.

The **agentic workflow** could define the broader process:

Issue created → Agent investigates → Agent proposes fix → Automated tests → Human review → Merge

Some stages are autonomous, while others are deterministic or require human approval.

The **Agentic AI system** is everything required to make this production-ready:

LLM + Agent + Workflow + GitHub integration + Repository + Test runner + Memory + Permissions + Security + Logging + Monitoring + Human approval + Evaluation

ANTHROPIC identifies five foundational patterns for constructing AI agents.

1. Prompt Chaining

Definition: Prompt chaining breaks a complex task into a **fixed sequence of smaller LLM calls**, where the output of one step becomes the input to the next.

Conceptually:

Input → LLM 1 → Output → LLM 2 → Output → LLM 3 → Final Result

For example, suppose we want AI to produce a technical article. The first LLM call could generate an outline, the second could validate the outline against requirements, and the third could write the final article.

The important characteristic is that the **sequence is predetermined by the developer**. The LLM performs individual steps, but it does not dynamically decide the overall workflow.

Prompt chaining is useful when a problem can be cleanly decomposed into predictable sequential stages. Anthropic notes that intermediate programmatic checks or “gates” can also be inserted between stages. [Anthropic](#)

2. Routing

Definition: Routing first **classifies an input and then sends it to the most appropriate specialized process, prompt, model, or tool.**

Conceptually:

Input → Router → Choose Appropriate Path

For example, consider a customer-support system:

Customer Query

- Billing issue → Billing workflow
- Technical issue → Technical-support workflow
- Refund request → Refund workflow
- General question → General QA workflow

Instead of using one large prompt to handle every possible situation, the router identifies what kind of problem it has received and sends it to a specialized component.

Routing can also be used for model selection. Straightforward questions could be sent to a smaller, cheaper model, while difficult problems could be routed to a more capable model.

The key idea is:

Different inputs require different processing strategies.

Routing works particularly well when the possible categories are sufficiently distinct and can be classified reliably. [Anthropic](#)

3. Parallelization

Definition: Parallelization divides work across **multiple independent LLM calls that execute simultaneously**, after which their results are combined or evaluated.

Conceptually:

Task

□ LLM A
↓ LLM B
□ LLM C

→ **Aggregate Results → Final Result**

Anthropic describes two important forms of parallelization: **sectioning** and **voting**. [Anthropic](#)

With **sectioning**, different LLM calls solve different parts of the problem. For example, when reviewing an application, one call might evaluate security, another performance, and another maintainability.

With **voting**, multiple LLM calls independently examine the same problem. Their outputs are then compared or aggregated. For example, several independent reviewers might analyze the same source code for vulnerabilities.

Parallelization is useful when subtasks are independent or when obtaining multiple perspectives improves confidence.

4. Orchestrator–Workers

Definition: In the orchestrator–workers pattern, a central **orchestrator dynamically determines how to divide a problem into subtasks, delegates those subtasks to worker LLMs, and combines their results.**

Conceptually:

User Task

↓

Orchestrator

□ ↓ □

Worker A Worker B Worker C

□ ↓ □

Orchestrator

↓

Final Result

For example, suppose the request is:

"Analyze this GitHub repository and determine why the application is slow."

The orchestrator might dynamically decide:

Worker 1: Analyze database queries.

Worker 2: Analyze API performance.

Worker 3: Analyze caching.

Worker 4: Analyze frontend network requests.

After the workers complete their tasks, the orchestrator synthesizes their findings.

The crucial distinction from ordinary parallelization is that **the subtasks are not necessarily predefined.**

With parallelization, a developer might explicitly define:

Security analysis + Performance analysis + Code-quality analysis

With orchestrator–workers, the orchestrator determines the necessary subtasks based on the particular problem it receives.

Anthropic specifically highlights this dynamic decomposition as the important distinction between orchestrator–workers and parallelization. [Anthropic](#)

5. Evaluator–Optimizer

Definition: The evaluator–optimizer pattern uses one LLM to **generate or improve a solution** and another LLM to **evaluate that solution and provide feedback**, creating an iterative refinement loop.

Conceptually:

Generator → **Solution** → **Evaluator**

↑ ↓

← **Feedback** ←

The process continues until the solution satisfies predefined criteria or a stopping condition is reached.

For example:

Coding LLM: Generate implementation.

Evaluator LLM: "Two unit tests fail and input validation is missing."

Coding LLM: Modify implementation.

Evaluator LLM: "Tests now pass, but error handling is incomplete."

Coding LLM: Improve error handling.

Evaluator LLM: "Requirements satisfied."

→ **Finish**

This pattern is particularly useful when there are **clear evaluation criteria** and iterative feedback can measurably improve the result. Anthropic gives examples including literary translation and complex search tasks. [Anthropic](#)

How the Five Patterns Differ

Pattern	Core idea	Structure
Prompt Chaining	Solve sequentially	$A \rightarrow B \rightarrow C$
Routing	Select the appropriate path	Input → Classify → A/B/C
Parallelization	Solve independent work simultaneously	$A + B + C \rightarrow \text{Aggregate}$
Orchestrator–Workers	Dynamically create and delegate subtasks	Orchestrator → Workers → Orchestrator
Evaluator–Optimizer	Iteratively critique and improve	Generate ↔ Evaluate

Agentic Systems

→ **Workflows**

- Prompt Chaining
- Routing
- Parallelization
- Orchestrator–Workers
- Evaluator–Optimizer

→ **Agents**

- Model dynamically determines its process
- Uses tools
- Observes environmental feedback
- Re-plans when necessary
- Continues until completion or a stopping condition

Key Implementation Details

- **LLM-Centric Design:** The LLM is the core “brain,” augmented with tools, retrieval systems and memory as needed.
- **Tool Integration:** Agents can access external tools (APIs, databases, actions) to retrieve information or execute tasks, such as issuing refunds or updating records in customer support scenarios.
- **Autonomy and Feedback:** Agents plan and execute actions, receive feedback from the environment (or users) and iterate until the task is complete or a stopping condition is met.
- **Human Oversight:** For critical or ambiguous steps, agents can pause for human input or review, ensuring reliability and safety.
- **Clear Interfaces:** Well-documented toolsets and explicit agent-computer interfaces (ACI) are essential for agent reliability and maintainability.